rmatch Status Report: 2026-03-03

Repository status snapshot

March 3, 2026

Abstract

This document captures the current state of the rmatch engine and its benchmark platform. It focuses on the present architecture, implemented optimization layers, current benchmark evidence, and the modeling work used to estimate regime dominance across pattern count and corpus size. It intentionally avoids the full historical journey and is structured as a reusable baseline for future publication work.

Contents

1 Scope and Data Sources **2**

2 System Architecture (Current) **2**
2.1 High-level execution model . . . 2
2.2 Compilation pipeline . . . 2
2.3 Matching pipeline . . . 2
2.4 Implemented optimization layers . . . 3

3 Benchmark Platform and Fairness Controls **3**
3.1 Canonical comparator set . . . 3
3.2 Fairness and reproducibility controls in place . . . 3
3.3 Known limitations to disclose . . . 4

4 Current Benchmark Evidence **4**
4.1 Primary comparable cohort . . . 4
4.2 Combined workload results (canonical + 10K partial) . . . 4

5 Runtime Modeling and Dominance Regions **5**
5.1 Model form . . . 5
5.2 Current fitted coefficients . . . 5
5.3 Dominance map figure . . . 5
5.4 Interpretation . . . 5

6 Memory and Correctness Status **5**
6.1 Memory telemetry . . . 5
6.2 Correctness consistency tracking . . . 7

7 External Baseline Context **7**
7.1 Why java-native-naive is kept 7
7.2 Why RE2J is kept 7
7.3 About very high-performance native engines 7

8 Cost and Operations Snapshot for Outsourced Runs **7**

9 Candidate Optimization Backlog (Not Yet Executed) **7**

10 Conclusion **8**

1 Scope and Data Sources

This report is built from repository-local artifacts as of 2026-03-03, primarily:

- engine/runtime code in `rmatch/src/main/java/no/rmz/rmatch`
- benchmark control and protocol docs in `extended-benchmarking/regex_bench_framework/docs`
- aggregated benchmark outputs in `extended-benchmarking/regex_bench_framework/reports/workload_a`
- model and dominance note in `.../workload_all_live/modeling_dominance_note.tex`

This is a status document, not a final publication. Results should be treated as current evidence with explicit caveats.

2 System Architecture (Current)

2.1 High-level execution model

rmatch is a non-backtracking matcher built around compiled NDFA fragments and lazily materialized DFA states. The default entry point (`MatcherFactory.newMatcher()`) instantiates a `MultiMatcher` that partitions regexes across sub-matchers and executes those sub-matchers in parallel.

2.2 Compilation pipeline

- `NDFACompilerImpl` compiles a regex string through `SurfaceRegexpParser` and `ARegexpCompiler`.
- `ARegexpCompiler` builds NDFA fragments (alternatives, char sets, quantifiers, any-char, etc.) and returns an NDFA start node.
- `NodeStorageImpl` maintains start-node connectivity and performs subset-construction mapping from NDFA sets to DFA nodes.

2.3 Matching pipeline

- `MatcherImpl` owns `RegexpStorage` and one `MatchEngine`.
- engine selection is property-driven (`rmatch.engine`), currently defaulting to `fastpath`.
- `MultiMatcher` shards regexes by hash and runs all shards concurrently on input clones.

- `NodeStorageImpl` and `DFANodeImpl` cache transitions and lazily create new DFA states as needed.

2.4 Implemented optimization layers

The current codebase contains several active optimization tracks:

- **FastPath engine**: ASCII fast-lane (`AsciiOptimizer`), thread-local state buffers (`StateSetBuffers`), and Aho-Corasick literal prefilter integration.
- **Prefiltering**: literal extraction (`LiteralPrefilter`) and candidate start-position narrowing via `AhoCorasickPrefilter`.
- **Memory/state representation**: compressed DFA basis representation (`CompressedDFAState`) with precomputed hash and integer-array storage.
- **Parallel partitioning**: default partition count heuristics in `MatcherFactory` and concurrent execution in `MultiMatcher`.

3 Benchmark Platform and Fairness Controls

The current benchmark control plane uses the Python framework under `extended-benchmarking/regex_bench_fr` containerized execution, and GCP run control.

3.1 Canonical comparator set

The active canonical set is:

- `rmatch`
- `re2j`
- `java-native-naive` (`java.util.regex`-based baseline)

3.2 Fairness and reproducibility controls in place

From `FAIRNESS_AND_REPRODUCIBILITY.md`, the current process enforces:

- fixed comparator set and standardized metric parsing contract
- canonical matrix and deterministic data inputs
- versioned dataset bundles with content hash
- transfer/setup excluded from timed benchmark metrics
- containerized runtime with architecture/cohort labeling
- run provenance artifacts, periodic checkpoints, and resumable run ledger (`jobs.db`, transaction logs)
- explicit hard timeout and stall timeout policy with persisted failure evidence

3.3 Known limitations to disclose

Current limitations called out by the repo protocol include warmup semantics not yet fully executed in runner logic, non-randomized run order, incomplete manifest hashing/commit capture, and regex semantic drift risk across engines.

4 Current Benchmark Evidence

4.1 Primary comparable cohort

Primary comparisons in this report use cohort `e2-standard-8|x86_64`, from:

- `cohort_workload_engine_matrix.csv`
- `all_runs_workload_engine.csv`

4.2 Combined workload results (canonical + 10K partial)

Patterns	Input	Winner	Winner ms	Java x	RE2J x	rmatch x
10	1MB	java-native-naive	125.0	1.00	2.83	52.13
10	10MB	java-native-naive	254.4	1.00	1.73	31.29
10	100MB	re2j	1072.4	1.41	1.00	20.96
100	1MB	java-native-naive	1692.4	1.00	3.61	4.09
100	10MB	rmatch	11175.4	1.21	5.03	1.00
100	100MB	rmatch	51477.8	2.62	10.91	1.00
1000	1MB	java-native-naive	15244.8	1.00	3.81	1.09
1000	10MB	rmatch	18382.5	8.97	31.51	1.00
1000	100MB	rmatch	41215.5	36.69	139.98	1.00
10000	1MB	rmatch	17340.1	7.92	14.74	1.00
10000	10MB	rmatch	19762.0	61.02	–	1.00
10000	100MB	rmatch	63219.3	–	406.78	1.00

Table 1: Comparable cohort summary (`e2-standard-8|x86_64`) combining canonical and current 10K rows. Multipliers are relative to the winner for that row (1.00 is best). "–" denotes currently missing data.

Winner count on these nine canonical points is:

- java-native-naive: 4
- re2j: 1
- rmatch: 4

Failure evidence currently captured: `all_runs_failed_jobs.csv` contains two logged failures for `re2j` at 10K/10MB (durations about 2543–2576 seconds) in this campaign line.

5 Runtime Modeling and Dominance Regions

5.1 Model form

The current fitted model uses a log-linear relation:

$$\ln T = \beta_0 + \alpha \ln P + \beta \ln B$$

where:

- T: runtime (ms)
- P: pattern count
- B: input size (bytes)

Anchored form used in the report artifacts:

$$T \approx K_{100,10\text{MiB}} \left(\frac{P}{100}\right)^{\alpha} \left(\frac{B}{10\text{MiB}}\right)^{\beta}$$

5.2 Current fitted coefficients

Engine	N	$K_{100,10\text{MiB}}$ ms	α	β	R^2
java-native-naive	9	9384.7	1.316	0.830	0.970
re2j	10	22263.6	1.289	0.851	0.888
rmatch	12	15357.9	0.142	0.294	0.851

Table 2: Fitted coefficients from `modeling_dominance_note.tex` for cohort `e2-standard-8|x86_64`.

5.3 Dominance map figure

5.4 Interpretation

Current fit suggests much weaker pattern/input exponents for rmatch than for java-native-naive and re2j in this cohort. This is directionally consistent with observed winner transitions: java-native-naive dominates at lighter workloads, while rmatch dominates more of the heavier-load region.

6 Memory and Correctness Status

6.1 Memory telemetry

The benchmarking API supports memory fields (`MEMORY_BYTES` and related fields), but current all-runs aggregation does not yet provide a clean, publication-grade memory series across campaigns/cohorts. In practical terms, the present reporting is runtime-focused; memory is an important missing benchmark dimension in the current campaign outputs. This gap is already recognized and is planned to be addressed in upcoming data test campaigns.

Interpretation. A practical dominance heuristic is to compare per-engine effective work terms $K_{100,10\text{MiB}}\, P^{\alpha} B^{\beta}$ and select the minimum.

2D Dominance Map

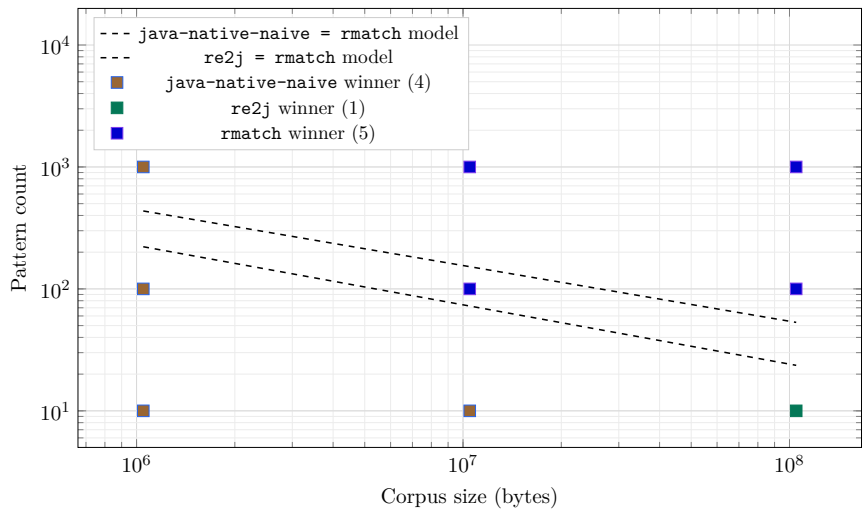

Figure 1: Observed winner per workload point (x = corpus size, y = pattern count) for scenarios with at least two selected engines present, with dashed model-equality boundaries ($T_i = T_j$).

Observed Scenario Winners

Patterns	Input (MiB)	Winner	Winner ms	Engines present
10	1.0	`java-native-naive`	125.029	`java-native-naive, re2j, rmatch`
10	10.0	`java-native-naive`	254.442	`java-native-naive, re2j, rmatch`
10	100.0	`re2j`	1,072.4	`java-native-naive, re2j, rmatch`
100	1.0	`java-native-naive`	1,692.4	`java-native-naive, re2j, rmatch`
100	10.0	`rmatch`	11,175.4	`java-native-naive, re2j, rmatch`
100	100.0	`rmatch`	51,477.8	`java-native-naive, re2j, rmatch`
1000	1.0	`java-native-naive`	15,244.8	`java-native-naive, re2j, rmatch`
1000	10.0	`rmatch`	18,382.5	`java-native-naive, re2j, rmatch`
1000	100.0	`rmatch`	41,215.5	`java-native-naive, re2j, rmatch`
10000	1.0	`rmatch`	17,340.1	`re2j, rmatch`

Figure 1: Dominance map and model-equality boundaries from the generated modeling note artifact.

6.2 Correctness consistency tracking

The consolidated consistency audit (`correctness_consistency_audit.csv`) currently reports:

- same-engine inconsistency: 5 minor, 4 major
- cross-engine mismatch: 6 minor, 3 major

These discrepancies are tracked and reported; runs are currently retained for performance analysis while investigation continues.

7 External Baseline Context

7.1 Why java-native-naive is kept

The native Java matcher is a practical baseline because it is mature, easy to integrate, and often excellent at small to moderate scales. It serves as a strong "small-load efficiency" reference.

7.2 Why RE2J is kept

RE2J provides a non-backtracking Java comparator in the same language/runtime ecosystem, making it operationally convenient and scientifically useful for scaling comparisons.

7.3 About very high-performance native engines

Repository benchmark harness documentation includes placeholders and plans for native engines such as Hyperscan. Those comparisons are important but are not part of the current canonical, publication-ready cohort in this report.

8 Cost and Operations Snapshot for Outsourced Runs

From the phased plan document, current planning envelope for one canonical campaign:

- warm run wall-clock: about 1.0–2.0 hours
- cold run wall-clock: about 2.0–3.5 hours
- example cost bands: roughly $0.50–$8.75 depending on VM hourly rate and warm/cold state

These are planning estimates only and must be replaced with current GCP region/machine pricing before publication claims.

9 Candidate Optimization Backlog (Not Yet Executed)

Current TODO/proposal themes relevant to performance work:

- stronger cutoff heuristics based on non-match termination statistics
- deeper investigation of match-count discrepancies and shutdown/flush timing
- engine-level match checksums for stronger cross-engine equality checks
- run-level memory telemetry integration into reports
- continued benchmarking-method cleanup and script hardening

10 Conclusion

The system now has:

- a concrete non-backtracking multi-pattern engine with parallel partitioning and active optimization layers
- a cloud-ready, resumable, and fairness-documented benchmark control plane
- a current empirical baseline with explicit workload-dependent winner regions
- an initial model-based dominance map suitable for guiding next optimization probes

Primary near-term priorities are to complete missing heavy-load cells, improve memory observability, and close correctness-discrepancy diagnostics before claiming publication-grade finality.